

Circular linked list

A circular linked list is a sequence of elements in which every element has link to its next element in the sequence and the last element has the link to the first element in the sequence.

Application of circular linked list

- (i) It is used by the operating system to share time for different user. It generally uses a round-robin time sharing mechanism.
- (ii) Implementation of advanced data structure like fibonacci heap.
- (iii) It is used in applications where the entire list access one-by-one in a loop.

Time complexity of linked list:-

(i) Traversal - Traversing a linked list means we visit each element in sequence where each node requires a constant time.

Since, linked list contains n -nodes, the total time required to traverse the list is $O(n)$.

(ii) Insertion -

a) Insert at first: Only one element then it has constant time. Hence its complexity is $O(1)$.

b) Insert at any position: T.C = $O(n)$

c) Deletion of nodes in linked list:

First delete = $O(1)$

Last = $O(n)$

Tree data structure

- Tree data structure is a collection of nodes which is organised in a hierarchical structure recursively.

- In tree data structure each individual element is called a node which contains the data and link to the next element in a hierarchical structure.

- If a tree has n nodes then the maximum number of edges is $n-1$.

Root - Root is the first node of any tree data structure. It is the origin of the tree, in any tree there must be exactly one root node.

Edge - Edge is the link between two nodes.

Parent node - Parent node is the predecessor of any node in a tree called a parent node.

Child node - Any successor of node is called child node.

Sibling - In tree data structure nodes which belong to same parent is called sibling node.

Leaf node - In tree data structure the node which has not any children is called leaf node.

Internal node - In tree data structure the node has atleast one child node is called internal node. Root node is also ~~called~~ internal node. It is also called non-terminal node.

Degree - The total numbers of children for any node is called its degree.

The highest degree of a node ^{among} ~~among~~ all the node in a tree is called degree of a tree.

Level ^{tree} **Label** - In a data structure each step from top to bottom is called as a ^{level} label and its start from root node having a ^{level} label 0.

Height - Total numbers of edges from leaf node to a particular node in the longest path is called height of the node. In a tree, height of the root node is said to be height of the tree.

Depth - Longest path from root node to leaf node is called Depth.

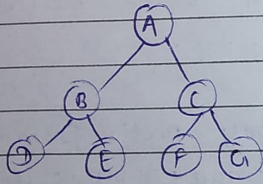
Sub-tree - In a non-linear tree data structure each child from a node form a subtree recursively.

Binary tree

The special type of tree where each node contain maximum numbers of two child node is called binary tree.

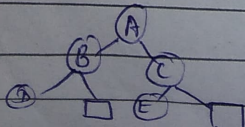
Strictly Binary tree

Special type of binary tree where each node contain two child node or zero children is called strictly binary tree.



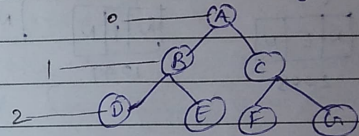
Extended Binary tree

This is the special type of binary tree where we adding dummy nodes to make complete binary tree is called extended binary tree.



complete Binary tree

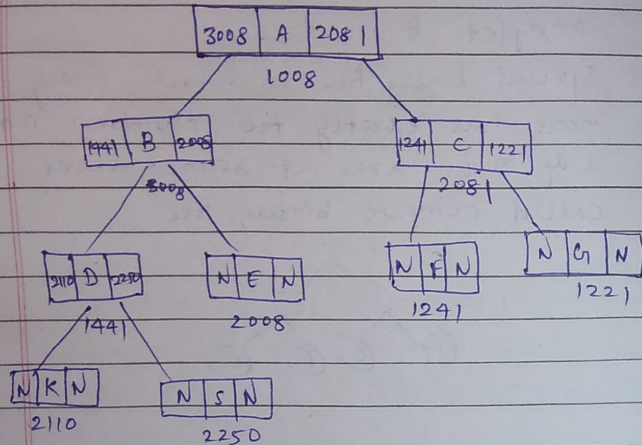
Special binary tree in which every internal node has exactly two children and all leaf nodes are at same level is called complete binary tree.



Representation of Binary tree

We represent any binary tree with the help of arrays and linked list. In linked list we represent each node with 3 fields. First field is reference field, 2nd field is data field and 3rd field is also reference field. Every node in binary tree having maximum numbers of children two where 1st reference field contain the address of left child and 2nd reference field contain the address of right child.

In binary tree, we use doubly linked list.



Array representation of binary tree
we represent any binary tree with the help of array if depth of tree is N then we needed ~~2N~~ $2N+1$ arrays.

Tree traversal

Tree traversal is the process in which we visit all the nodes included in tree.

There are three types of traversal —

- (i) InOrder
- (ii) PreOrder
- (iii) Post Order

InOrder Traversal

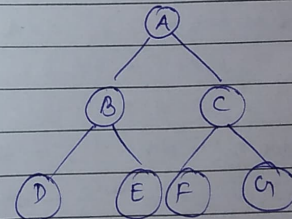
In this type of tree traversal first we visit left node then root node and last right node.

Pre Order Traversal

In this tree traversal first we visit the root node then left child and last right child.

Post Order Traversal

In this tree traversal first we visit the left node then right node and last root node.



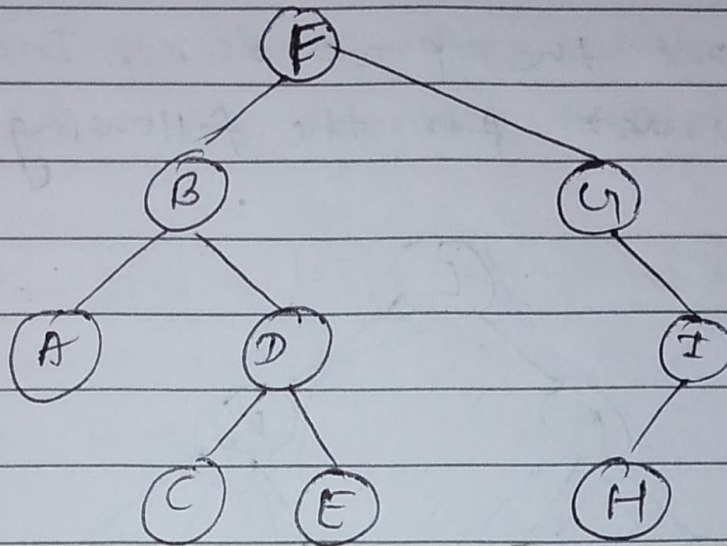
~~Pre~~InOrder

D - B - E - A - F - C - G

Pre-Order = A

Post-Order =

Ques.



Pre order - F-B-A-D-C-E-G-I-H

In order - A-B-C-D-E-F-G-H-I

Post order - A-C-E-D-B-H-I-G-F

Pre - Root $\rightarrow$ L $\rightarrow$ R

In order - L $\rightarrow$ Root $\rightarrow$ R

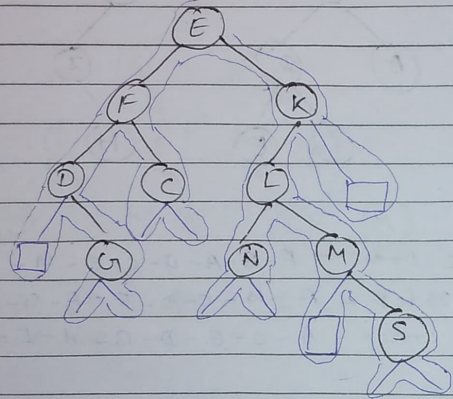
Post - L $\rightarrow$ R $\rightarrow$ Root

node $\rightarrow$ 1st time $\rightarrow$ pre

node $\rightarrow$ 2nd time $\rightarrow$ Inorder

node $\rightarrow$ 3rd time $\rightarrow$ Post

Ques. Find out the preorder, Inorder and post order for the following binary tree.



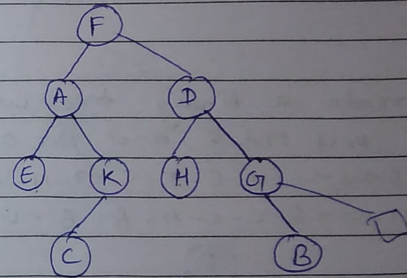
Pre-Order - E-F-D-G-C-K-L-N-M-S

InOrder - D-G-F-C-E-N-L-M-S-K

Post Order - G-D-C-F-N-S-M-L-K-E

Creation of binary tree with the help of post order and Inorder traversal, Pre order

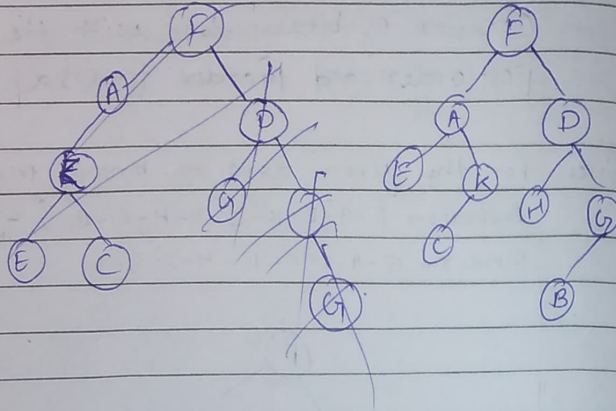
Ques. For the given data of binary tree traversal,
PreOrder → F-A-E-K-C-D-H-G-B
InOrder → E-A-C-K-F-H-D-B-G } create a binary tree.



Ques. Create a binary tree from the following traversal:

Post Order → E-C-K-A-H-B-G-D-F

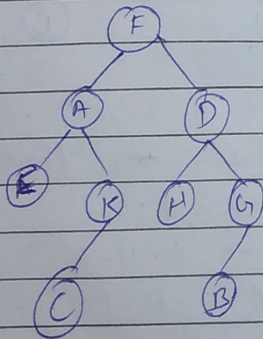
InOrder → E-A-C-K-F-H-D-B-G



Ques. Create a binary tree with the help of post order and pre order.

PreOrder $\rightarrow$ F-A-E-K-C-D-H-G-B

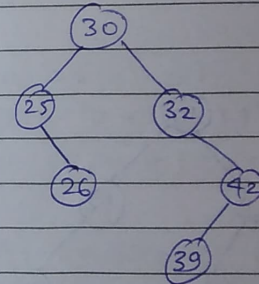
PostOrder $\rightarrow$ E-C-K-A-H-B-G-D-F



BST (Binary Search Tree)

BST is a non-linear data structure that store elements in a sorted order. And Searching insertion and deletion operation are easy to other binary trees.

In this tree, every node value is greater than its left child and less than its right child.

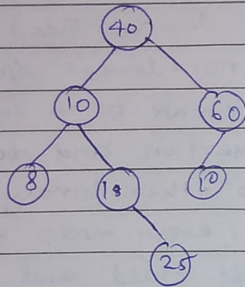


Ques. Create a BST for the following data.

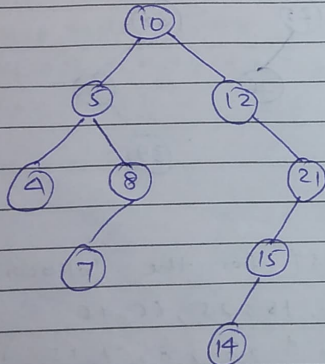
(i) 40, 10, 8, 18, 25, 60, 10

(ii) 10, 12, 5, 4, 21, 8, 7, 15, 14

(i)

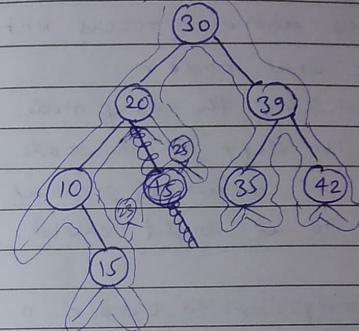


(ii)



(iii)

30, 20, 10, 15, 25, 23, 39, 35, 42



Pre-order $\rightarrow 30 - 20 - 10 - 15 - 25 - 23 - 39 - 35 - 42$

Post-order $\rightarrow 15 - 10 - 23 - 25 - 20 - 35 - 42 - 39 - 30$

In-order $\rightarrow 10 - 15 - 20 - 23 - 25 - 30 - 35 - 39 - 42$

Algorithm for creation of BST (Insertion)

Step 1:- We create a node with data and two reference field NULL.

Step 2:- Check the BST is empty or NOT.

Step 3:- If root is not empty then assign the new node to root node i.e. $root = newnode$.

Step 4:- If root is not empty then check

(i) if value of $newnode \leq root$ node then move to its left child.

(ii) if value of $newnode > root$ node then move

to its right child.

Step 5:- Repeat the above process until we reach the leaf node.

Step 6:- After reaching the leaf node insert the new node in left if, $\text{newnode} < \text{leaf node}$ otherwise if $\text{newnode} > \text{leaf node}$ then its shift to its right child.

Ques. write a program to create a BST and display its content.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
int data;
```

```
    left    right  
    struct node * prev, * next;
```

```
};
```

```
void inorder(struct node * root);
```

```
void preorder (struct node * root);
```

```
void postorder (struct node * root);
```


else

{

~~PreOrder~~ PostOrder(left → data);

PostOrder(right → data);

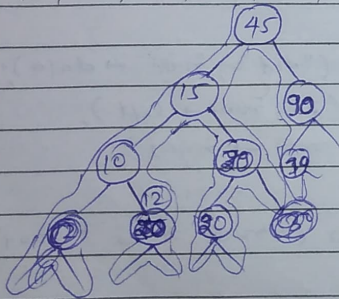
Printf("%d", root → data);

}

}

Ques. Write a program in C to create binary search tree and traverse with three method preOrder(), postorder(), InOrder().

Ques. Create a binary ^{search} tree for the following data:
45, 15, 79, 90, 10, 55, 12, 20, 30



write down the preorder postorder for the given BST.

PreOrder → 45-15-10-12-79-20-30-90

PostOrder → 12-20-30-15-45

InOrder → 10-12-15-20-30-45-79

09/07/20

Deletion in binary search tree

When we delete any node from binary search tree, first we maintain its properties with the help of three cases :-

Case 1 - If the node is leaf node simply we remove the node from the tree by setting the reference field to NULL.

Case 2 - If node has only one child then the parent of the deleted node should be connected to directly to child node of the deleted node

Case 3 - If the node has two children, this is the complex method in BST to remove the concerning node. For this we find out the inorder predecessor or

inorder successor in existing BST.

Inorder predecessor

In BST we find the inorder predecessor for searching the largest value in left subtree.

Replace this value with the node which we want to remove in BST.

Inorder Successor

In this method we find out the largest element in right subtree and replace this value with the node which we want to remove from the BST.

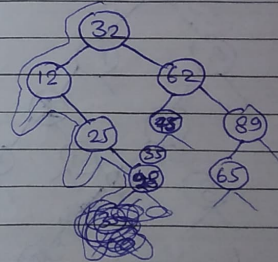
Ques. what is BST? Discuss the insertion and deletion steps in the following data.

Ques. The preorder traversal sequence of BST is 30, 20, 10, 15, 25, 23, 39, 35, 42. Write step-by-step process to derive the BST and find postorder traversal also.

In Inorder traversal the resultant nodes are in increasing order.

Ques. Create BST for the following data and find out preorder, inorder and postorder traversal.

32, 62, 12, 25, 28, 45, 89, 65, 35



Pre Order = 32-12-25-28-35-45-35-89-65

Post Order = 28-25-12-35-65-89-62-32

Inorder = 12-25-28-35-45-62-65-89

AVL tree

AVL tree is balancing BST where the balance factor for every node may be -1, 0 and 1.

Those trees whose all leaf nodes are at same level is called balancing tree.

In AVL tree, every node contain the value -1 or 0 or 1. If these values not for any node then tree is unbalanced.

Balance factor

This is the numerical value which we find by $R = (\text{height of left subtree} - \text{height of right subtree})$.

This balancing factor may be negative, zero or positive.

Balance property

This is the property of any node in AVL where each node balancing

factor is -1, 0 or 1.

- Time complexity of AVL is $O(\log n)$.

For any BST which is unbalanced convert to balanced BST (AVL) by performing some rotation. It may be clockwise or anticlockwise.

There are four types of rotation —

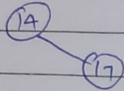
- (i) LL rotation — Anticlockwise
- (ii) RR rotation — clockwise
- (iii) LR rotation
- (iv) RL rotation

Ques. What is AVL tree? Discuss the different types of rotation in AVL tree or balancing the BST.

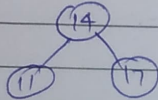
Ques. Construct an AVL tree from numerical value 1 to 8.

Ques. 14, 17, 11, 7, 53, 4, 13, 12, 8, 60, 19, 16, 21
Create AVL tree for this data and describe each steps.

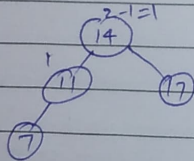
Step 1: Insert 14, 17



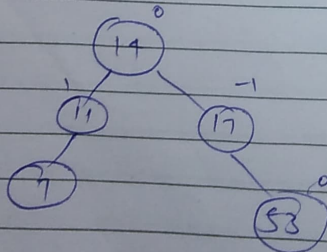
Step 2: Insert 11



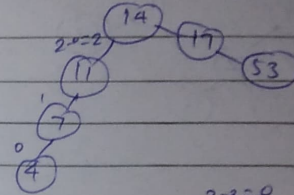
Step 3: Insert 7



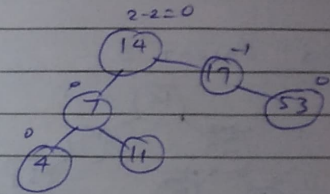
Step 4: Insert 53



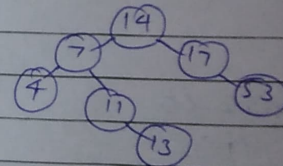
Step 5: Insert 4



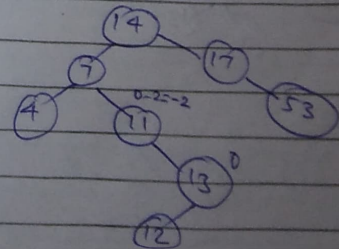
LL rotation



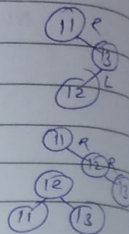
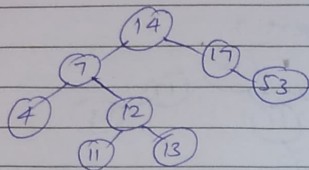
Step 6: Insert 13



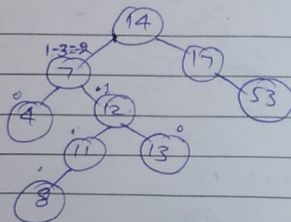
Step 7: Insert 12



RL rotation

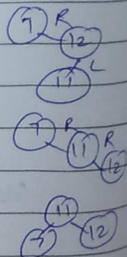
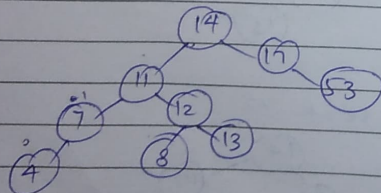


Step 8: Insert 8

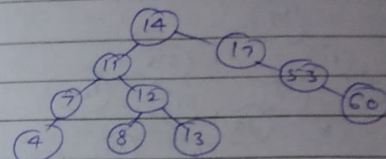


R-L rotation

Step 9: Insert 60

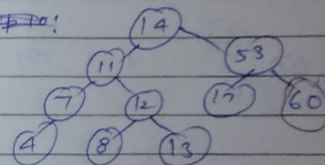


Step 9: Insert 60

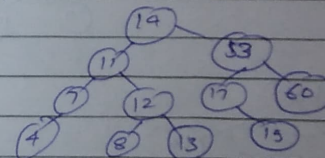


R R rotation

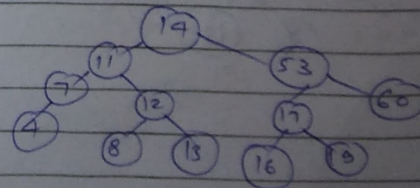
Step 10:



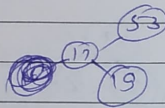
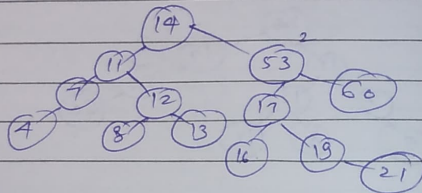
Step 10: Insert 19



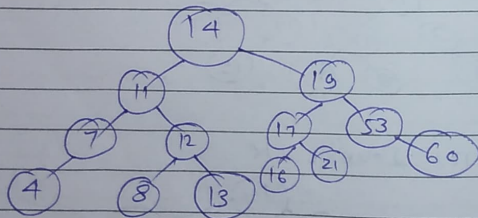
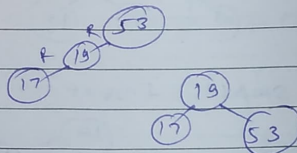
Step 11: Insert 16



Insert 21



L-R rotation

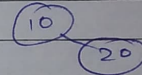


Ques. Insert the following sequence of elements in an AVL tree

(A) 10, 20, 15, 25, 30, 16, 18, 19

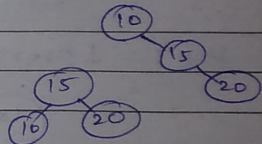
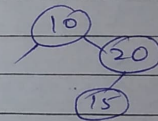
(B) Delete 30 in AVL tree.

Step 1: Insert 10, 20



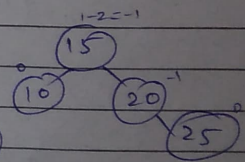
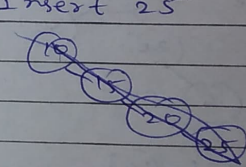
Step 2

Step 2: Insert 15

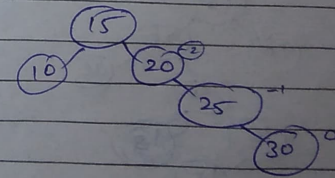


R-L rotation

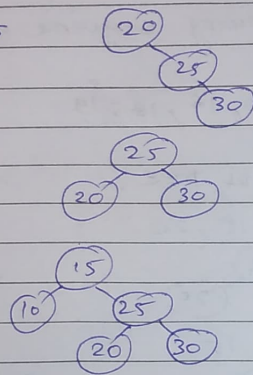
Step 3: Insert 25



Step 4: Insert 30

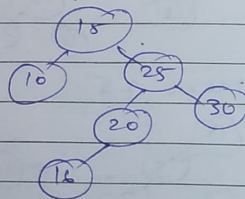


~~Step 5~~

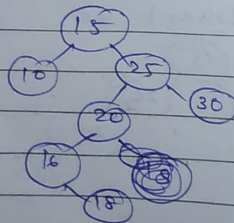


R-R rotation

Step 5: Insert 16



Step 6: Insert 18



B-Tree

B-tree is self-balancing search tree where each node can have more than two children and store multiple key values in a node.

This B-tree is used for efficient handle of large data set which stored in secondary storage devices. By using B-tree it minimize the number of disk access required for operation.

This is also extended of BST.

Properties of B-tree

- It is balance m way tree generalised BST in which a node can have more than one key and more than two children.
- If B tree of order m has following properties:-
 - (i) maximum no. of children is m.
 - (ii) minimum children
 - leaf node $\rightarrow 0$

- root node $\rightarrow 2$

- internal node $\rightarrow [m/2]$

(iii) maximum key of a node of order m is $m-1$ key.

(iv) minimum key

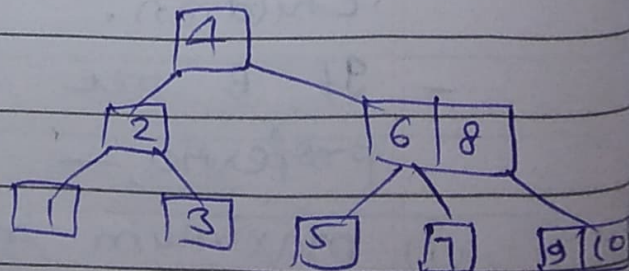
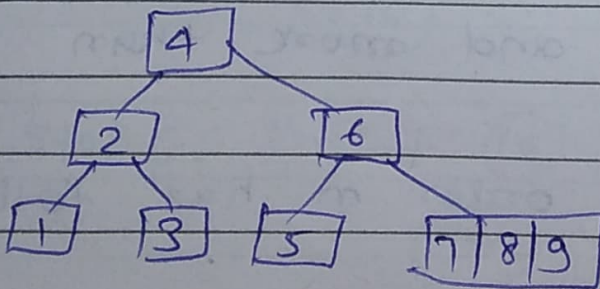
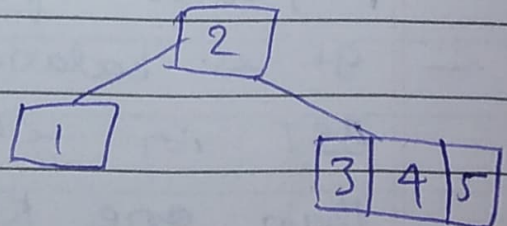
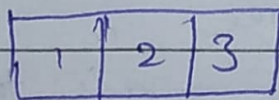
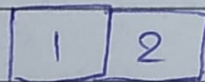
- root = 1

- another = $[m/2] - 1$

Ques. Create a B-tree of order 3 having element 1 to 10.

$$m=3$$

$$\text{maximum key} = 3-1 = 2$$



Ques. What is B+ tree? write down difference between B and B+ tree.

Ques. write the B tree of order 3 and order 4 created by inserting the following data arriving in sequence.

92, 29, 6, 7, 11, 8, 22, 4, 5, 16, 19, 20, 78

Ques. Draw the B tree of order 3 created by inserting data arriving in sequence from the two sets.

Compare the two B tree to determine the order of data will create different B tree.

Set 1 = 87, 78, 8, 9, 20, 33, 56, 49

Set 2 = 49, 56, 33, 20, 19, 8, 18, 89

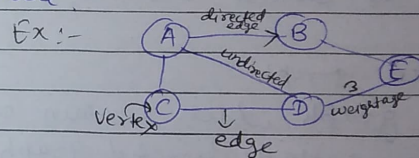
GRAPHS

Graph is a non-linear data structure which contain a set of points known as vertex and set of link known as edge which connect the vertices.

Graph Terminologies

(i) Vertex

An individual data element of a graph is called vertex. It is also known as node.



Edge

Edge is a connecting link between two vertex. There are 3 types of edges in graph.

(i) Directed edge

A directed edge is a unidirectional edge from any starting node to its adjacent node.

(ii) undirected edge

It is a bidirectional edge

(iii) weightage edge

Any edge which contains its numerical value is called weighted edge

Types of Graph

(i) undirected graph

Any graph which contains only undirected edges is called undirected graph.

(ii) Directed graph

These graph which contains only directed edge is called directed graph.

(iii) Complete graph

A graph in which any node is adjacent to all other node present in graph is known as complete graph.

(iv) Regular graph

It is a graph in which nodes are

adjacent to each other that is each node is accessible to any other node.

(v) Cyclic graph

Syntax - `connection <name> = DriverManager.getConnection()`

— How to give query to a JDBC (Querying to a database)

Query: Particulars code which will be responsible to generate the database.

DSA

23/03/26

Graph is a combination of vertex and edges.

Graph traversal

24/03/26

It is a technique which is used for a searching vertex in a graph. It finds the edges to be used in the search process without creating loops.

There are two types of traversal in graph -

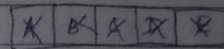
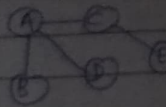
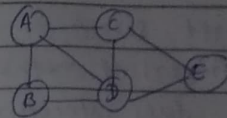
1. DFS (Depth First search)
2. BFS (Breadth First search)

After traversing by both methods the resultant graph is spanning tree i.e. graph without loops.

BFS - Queue Data Structure

DFS - Stack DS

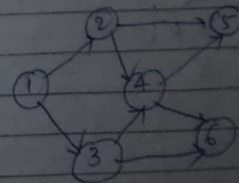
Ques.



Rules -

1. Define queue of size that is total no. of vertices exist in the graph.
2. Select any starting point for traversal.
3. Visit all its adjacent nodes that are not visited and insert in the queue.
4. If there are no more vertices from the vertex which is at the front of queue then delete the vertex from the queue.
5. Now repeat the step 3 and 4 until queue becomes empty.
6. After this traversing the resultant graph is spanning tree.

Ques. Write down the BFS traversal and DFS traversal of the given taking as source vertex.



Construct the B tree with minimum degree 3, 10 to 99

DFS - Depth First Search

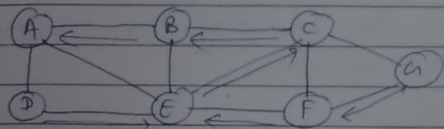
DFS is graph traversing method which is used in stack data structure to create a spanning tree.

Back Track

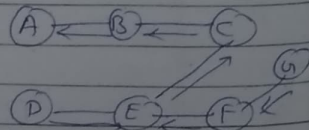
Steps -

- (i) Define stack of size = numbers of node in graph.
- (ii) Choose any starting point that is node for traversal.
- (iii) Visit any of the non-visited adjacent vertex of a vertex which is at the top of stack and push it on to the stack.

First we insert the starting node in a stack and other after it.



E
X
E
E
E
A



Ques. What is Graph? Discuss the steps involved in Traversing of any graph with BF and DFS.

Ques. Write down the difference between BFS and DFS.

Heap sort

Heap is a special binary tree based data structure that satisfy the heap property.

Heapify : It is the process to rearrange the binary tree to maintain the heap property after insertion or deletion of a node.

Heap sort : It is a sorting algorithm used to arrange the elements in ascending / descending order. It uses heap tree for sorting element by maximum heap or minimum heap.

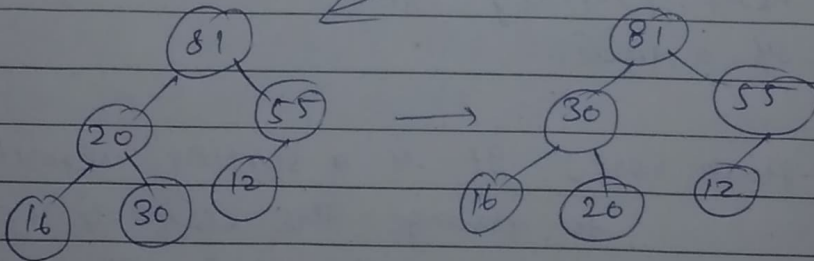
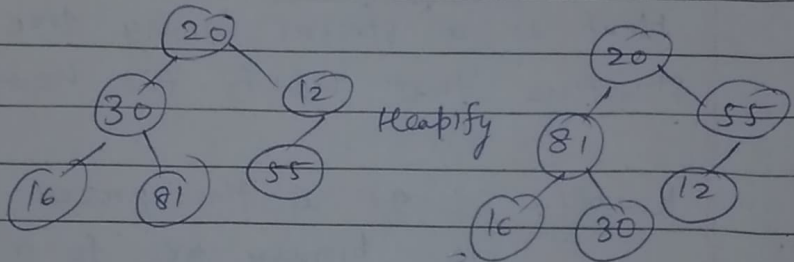
Process -

- (i) Construct a binary tree with the given element.
- (ii) Transform binary tree to minimum heap or maximum heap.
- (iii) Delete the root node / element from minimum maximum heap using heapify method.
- (iv) Put deleted elements that is root element in sorted list.

- (v) Repeat the process until minimum/maximum heap become empty.
- (vi) Display the sorted list.

Minimum heap $\rightarrow$ Ascending
Maximum heap $\rightarrow$ Descending

20, 30, 12, 16, 81, 55



81 - 55 - 30 - 20 - 16 - 12

